

Module 4 – Hands-on Sessions

CS5013 – Programming with AI
Debugging and Refactoring with AI

V. Krishna Nandivada
Department of Computer Science and Engineering, IIT Madras

Overview

- **Session 4A – AI-assisted debugging.** Reproduce a bug from a stack trace, use AI to explain, use AI to propose a patch, verify with tests.
- **Session 4B – Reducing cyclomatic complexity.** Take a legacy method and refactor it, with AI as the junior pair-programmer, until SpotBugs and the cyclomatic checker are both satisfied.

Starter contents (m4-pub/)

- `PriceEngine.java` – a ~90-line class with a method `quote(Order, Customer): Money` of high cyclomatic complexity and one latent bug.
- `PriceEngineTest.java` – JUnit 5 tests that exercise most of the code paths. Passes on `main`.
- `Order.java`, `Customer.java`, `Money.java` – supporting types.
- `Makefile` – pre-configured with JUnit 5, [Optional: TODO: Add SpotBugs plugin, and PMD plugin].

1 Session 4A: AI-assisted debugging

Objectives

By the end of this session you will be able to:

1. Reproduce a failing test on the branch.
2. Use an AI chat panel to explain a stack trace in your own words.
3. Ask the AI to rank plausible root causes and propose a minimal patch.
4. Verify the patch by re-running the test suite.
5. Commit with a hand-written “why” line.

Setup

1. Create a git-hub repository called m4-hands-on. Initialize the repo with the unzipped/un-tarred code from <https://www.cse.iitm.ac.in/~krishna/cs5013/m4-pub.tgz>.
2. `cd m4-pub`.
3. Add it to your git-repo.
4. Open the folder in VS Code (or IntelliJ IDEA).
5. Confirm Copilot / Claude Code is active in the editor.
6. Run `make test`. One test fails – `quotesDiscountForLoyalCustomer`. This is the bug you will debug in Part A.

Part A – Explain the stack trace

1. Look at the full stack trace from the make output.
2. Open the AI chat panel.
3. Paste the parts of stack trace (starting with just the first line, then two lines, then three lines, and so on) with the following prompt (do not change the wording):

Explain **this** Java stack trace in plain English. What is the most likely root cause? Do NOT propose a patch yet.

4. Read the reply.
5. In your `PROMPTS.md`, record the AI's explanation verbatim, when you get the right solution, then a two-line note in your own words about whether you agree, and if you need the full stack trace to be pasted always.

Part B – Propose and verify the patch

1. Ask:

Given root cause X (paste the diagnosis from previous Part), write the smallest patch that fixes the bug without changing behaviour on any other test. Show only the diff.

2. Read the patch *before* applying it. Does it change any behaviour outside the failing case?
3. Apply. Run `make test`.
4. If the failing test now passes and no other test regressed, proceed. If any test regressed, revert and re-ask with the regression as new context.
5. Commit with the AI-drafted commit message plus a hand-written “why” line. Use `git commit -e` to edit before committing.

Part C – Reflect

In `PROMPTS.md`, answer in 3–5 sentences:

- Which of the AI's replies (explain, patch) was the most useful? Which was the least useful?
- How much do you think should be given to the AI, in terms of stack trace? What is the consequence of pasting the full stack trace? Or just one/two lines?

Deliverables

- `PROMPTS.md` containing the AI replies and the reflection paragraph.

2 Session 4B: Reducing cyclomatic complexity

Objectives

By the end of this session you will be able to:

1. Measure cyclomatic complexity with a tool.
2. Use AI to propose refactor moves that reduce it.
3. Distinguish safe refactors (behaviour-preserving) from unsafe ones.

Setup

1. Continue on the same branch as Session 4A (with the bug now fixed and committed).
2. (Optional) Install `pmd` and `SpotBugs`. You can ask agentic mode to install it.
3. (Optional) Run `make pmd` to see the PMD report. Locate the “CyclomaticComplexity” finding on `PriceEngine.quote`.
4. (Optional) Run `make spotbugs` to see the SpotBugs report. There will be at least two findings.
5. Note down the cyclomatic complexity value. See if your manual calculations match that of `pmd`.

Part A – Rank refactor targets

1. Open `PriceEngine.java`.
2. In chat, paste the whole file with:

Rank the methods in `this` file by refactoring priority.
For each, give:

- (a) current cyclomatic complexity estimate,
- (b) the single refactor move you would apply first
(extract-method, flatten-guard-clauses, replace-loop-with-pipeline, or rename),
- (c) one sentence on the expected complexity reduction.

Do not propose code yet.

3. Save the ranking in `PROMPTS.md`.

Part B – Apply one extract-method refactor

Take the top-ranked method (should be `quote`).

1. Identify a coherent block inside `quote` that could become a helper. A good target is the “loyalty-tier discount” block or the “bulk-order surcharge” block.
2. Ask:

Extract the loyalty-tier discount block into a **private** helper method `'applyLoyaltyDiscount(...)'`. Do NOT change **public** method signatures. Show the diff.

3. Read the diff. Watch specifically for:
 - Changed types in parameters or return.
 - Missing final modifiers.
 - Silently-lifted early returns.
4. Apply. Run `make test` – *all* tests must still pass.
5. (Optional) Re-run `make pmd`. The complexity value on `quote` must decrease.
6. Commit as one atomic commit.

(Optional) Part C – Fix a SpotBugs finding with AI

Pick one SpotBugs finding from the earlier report. Copy the rule name and the flagged code region.

1. In chat:

SpotBugs reports the finding [RULE_NAME] on the following code. Explain what the rule requires and show the smallest patch that satisfies it without changing behaviour. Show the diff.

2. Apply. Re-run `make spotbugs` and confirm the finding is gone.
3. Re-run `make test` to confirm no regression.
4. Commit.

Part D – Reflect

In `PROMPTS.md`, answer:

- By how much did the cyclomatic complexity drop?
- Did the SpotBugs finding fix require you to understand the rule, or was the AI's explanation enough to proceed?
- List one refactor the AI *proposed* but you *rejected*. Explain why.

The rejected-refactor bullet is the mark of a good software engineer. Not accepting every AI suggestion is a skill.

Deliverables

Push the branch (same one as Session 4A). Amend the PR to include:

- The two additional commits (refactor + SpotBugs fix).
- `PROMPTS.md` containing the ranking, the applied refactor, and the reflection.